

```
$bar = new class {
    public function bar() {
        return "foo";
    }
};
var_dump($bar,$ bar -> bar ());
```

5. The Integer Division Function

The Integer Division (**intdiv**) function is not much of a significant addition, however worth mentioning. This method promises to return an **int** value always, for all the division operations performed. A regular division operation generally leads to a **float** result being returned. So earlier, in case, if you had to round off your result to get the **int** value...this method is a life saviour for you.

```
var_dump (intdiv (4,3),(4/3) );
```

This piece of code would return **1** instead of **1.34**.

6. Enhanced CSPRNG Functions

CSPRNG stands for Cryptographically Secure Pseudo Random Number Generator. PHP 7 has brought in a set of CSPRNG to generate random numbers in a secure way. Though, PHP already had methods like **rand()** to generate random data, but these methods weren't secure. In PHP 7, a system interface is tied along with the operating system's random data generator and because of this the security of the random data generated through PHP is as secured as the operating system. This can be very useful in case of random passwords and salts. CSPRNG introduces two new functions - **random_int** and **random_bytes**.

random_int

The **random_int** method basically returns a random number between a maximum and minimum range specified by the user. Consider the example below -

```
random_int(15,20);
```

The above piece of code would generate a random number between 15 and 20, including the number 15 and 20.

random_bytes

The **random_bytes** method returns random byte (generally string) of a length specified by the user. Consider the example below -

```
random_bytes(5);
```

The above piece of code would generate a random byte of length 5.

7. Import from the same namespace

PHP 7 has brought along with it another great new feature - Group use Declaration. This feature is a bliss for all those who want to import multiple classes from the same namespace. The new features cut out wordiness, and lets you import same namespace classes inline, thus making your code look cleaner.

```
Use lib\namespaceA\[ClassA, ClassB, ClassC as C];
```

Consider the above line of PHP code which uses the Group use Declaration. The above line of code declares **ClassA**, **ClassB** and **ClassC** which belong to same namespace **namespaceA** in a single line of code.

Note: If you were to do the same in PHP version 5.6 or below, we would have to write the following piece of code -

```
use lib\namespaceA\ClassA;
use lib\namespaceA\ClassB;
use lib\namespaceA\ClassC as C;
```

Move on to PHP 7

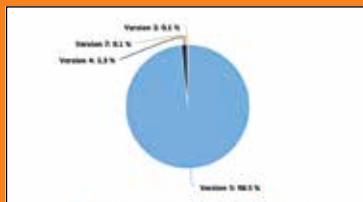
PHP 7 looks very promising. There are a lot of other features added in PHP 7 like Unicode support for international characters and emoji too! The performance improvement is tremendous and the security enhancement is remarkable. There are also a lot of features which have been removed in PHP 7. However, these features were already deprecated since PHP 5, so maybe you weren't using them at all. If your site is built using WordPress, there is a good news for you. WordPress Core is already getting ready for PHP 7. So, its high time you started considering upgrading your servers to PHP 7 for better performance and an easier life. >

*Facts about PHP



*Unlucky 6

>>Apparently, in the world of programming languages version 6 is considered to be highly unlucky. Eg - Perl 6, PHP 6 and MySQL 6.



*True majority

>>PHP powers 82.2% of all the websites which are dynamically programmed. Of these 80.72% of the websites are using PHP 5.



*Twice as fast

>>PHP 7 is 2x faster than PHP 5 i.e. it can handle twice as many requests per second. That's a 100% performance improvement.